Building a **Tool-Using Research Agent** in Python involves setting up a core loop where a Large Language Model (LLM) evaluates a research query, chooses to call a search tool (such as Tavily or SerpAPI), collects the raw output, and synthesizes it into a well-cited report.

Below is a complete, production-ready implementation using **LangChain** and a search API, structured around the ReAct (Reasoning + Acting) loop.

Step 1: Install Dependencies

Install the necessary libraries for model management, agent execution, and web searching:

Step 2: Set Up Environment Variables

Create a `.env` file in your project root directory and add your API keys:

Step 3: Complete Python Implementation

```
import
from        import
from                  import
from                      import
from                  import
from                        import

from        import

# 1. Load environment variables


# 2. Define the Search Tool
# Using Tavily, which is explicitly optimized for LLM research agents
                                              "TAVILY_API_KEY"

@tool
def web_search        str
    """
    Search the web for factual information regarding a given query.
    Returns a string containing snippets and source URLs.
    """
          f"\n[Tool Execution] Searching the web for: '{query}'..."
    try
                                                                  4

        for        in              "results"
```

```
                f"Title: {result.get('title')}\n"
                f"URL: {result.get('url')}\n"
                f"Snippet: {result.get('content')}\n"

        return "\n---\n"
    except           as
        return f"Error executing search: {str(e)}"



# 3. Initialize the LLM
                      "gpt-4o"              0

# 4. Design the Agent Prompt with Citation Directives


        "system"
        "You are an expert autonomous research assistant. Your task is
to thoroughly answer "
        "the user's research question by gathering data via the
`web_search` tool.\n\n"
        "Guidelines:\n"
        "1. Break complex questions down, search multiple times if
necessary, and cross-reference facts.\n"
        "2. Synthesize your final findings into a comprehensive,
professional report.\n"
        "3. **Mandatory Citation Rule:** You must explicitly cite your
sources using inline Markdown "
        "links (e.g., [Source Title](URL)) corresponding to the URLs
returned by the search tool.\n"
        "4. Include a dedicated 'References' section at the very end
listing all URLs used."

                                        "chat_history"
     "human"  "{input}"
                                        "agent_scratchpad"


# 5. Create the Agent and Executor



            True  # Prints out the agent's thought process and tool
calls
                        True


# 6. Run the Research Agent
if              "__main__"
                     "What are the latest breakthroughs in quantum
computing hardware for 2026?"

          f"Starting Research Task: '{research_query}'\n"   "=" 50


        "input"
```

```
    "chat_history"


      "\n"   "=" 50
      "FINAL RESEARCH REPORT:\n"
             "output"
```

How This Architecture Works

1. **The Planning & Reasoning Loop (`AgentExecutor`)**: When given a complex prompt, the LLM doesn't just guess; it outputs a tool-call request specifying what query to pass to `web_search`.

2. **Tool Integration (`@tool`)**: The `web_search` function safely executes an external API call via Tavily, formatting raw HTML/JSON snippets into clean text containing both content and metadata links.

3. **Observation & Refinement**: The agent reads the search response, analyzes gaps, and may choose to execute iterative searches (e.g., drilling down into a specific hardware provider discovered in search #1).

4. **Enforced Citations**: The system prompt explicitly mandates inline URLs and a reference log, ensuring that every claim made in the report maps back to a verifiable source.

Would you like to extend this script to incorporate **multi-agent delegation** (e.g., splitting duties between a primary researcher agent and a writer agent)?